

Introduction to Reinforcement Learning (RL) 🚁

Reinforcement Learning is the third pillar of Machine Learning (alongside Supervised and Unsupervised Learning). While it currently has fewer commercial applications than supervised learning, it is incredibly powerful for tasks involving control and decision-making.

The Core Concept: Training a Dog 🐶

The best way to understand RL is to think about training a puppy.

- You cannot explicitly tell a puppy "Move your left paw 3 inches forward."
- Instead, you watch the puppy behave.
- **Positive Reward:** If it sits, you say "Good Dog!" (+1 Reward).
- **Negative Reward:** If it chews the shoe, you say "Bad Dog!" (-100 Reward).
- Over time, the puppy learns to do more of the things that get rewards and fewer of the things that get punishments.

Reinforcement Learning works the same way: You don't tell the computer *how* to do a task; you just tell it *what* a good outcome looks like (the reward).

Example: The Autonomous Helicopter

Imagine you want to program a helicopter to fly upside down (an autonomous stunt).

- **The Problem with Supervised Learning:**
 - To use Supervised Learning, you would need a dataset of (x, y) .
 - x = Helicopter state (position, speed, orientation).
 - y = The *perfect* control stick movement.
 - **Issue:** It is extremely difficult (even for experts) to define exactly what the "perfect" stick movement is for every possible millisecond of flight.
 - **The Reinforcement Learning Approach:**
 - **State (s):** The helicopter's current position and orientation.
 - **Action (a):** How to move the control sticks.
 - **Reward Function:**
 - Flying stable? **+1 Reward** (Good Helicopter!)
 - Crashed? **-1000 Reward** (Bad Helicopter!)
 - **Result:** The algorithm experiments and learns a function (policy) that maps states (s) to actions (a) to maximize the total reward.
-

Applications of Reinforcement Learning

1. Robotics:

- Getting a robot dog to climb over complex obstacles.
- Autonomous flight (helicopters, drones).
- Landing a spacecraft (You will do this in the practice lab with a Lunar Lander!).

2. Game Playing:

- Chess, Go, Checkers.
- Video games (e.g., teaching AI to play Mario or Dota).

3. Industrial & Financial Optimization:

- **Factory Optimization:** Maximizing throughput and efficiency.
 - **Stock Trading:** Determining the optimal strategy to sell large amounts of stock over time without crashing the price.
-

Summary

- **Supervised Learning:** You tell the computer exactly what to do (State $x \rightarrow$ Correct Action y).
- **Reinforcement Learning:** You tell the computer the *goal* via rewards (State $s \rightarrow$ Action $a \rightarrow$ Reward or Punishment). The computer figures out the "how" by itself.

The Mars Rover Example

We use a simplified example inspired by the Mars Rover to formalize the reinforcement learning problem.

The Setup

- **States (s):** There are 6 possible positions (boxes) for the rover.
 - $s_1, s_2, s_3, s_4, s_5, s_6$.
 - The rover starts at a specific state (e.g., s_4).
 - **Rewards ($R(s)$):** Each state has an associated reward value.
 - s_1 : **+100** (Highly interesting science target).
 - s_6 : **+40** (Moderately interesting science target).
 - s_2, s_3, s_4, s_5 : **0** (No significant science value).
 - **Terminal States:** Once the rover reaches state 1 or state 6, the "day" ends. It collects the reward and stops moving.
 - **Actions (a):** At each step, the rover can move:
 - **Left** ($\leftarrow$)
 - **Right** ($\rightarrow$)
-

The Reinforcement Learning Loop

At every time step, the following sequence occurs:

1. **Current State (s):** The robot is in a specific position (e.g., state 4).
2. **Reward ($R(s)$):** The robot receives the reward associated with the *current* state (e.g., 0).
3. **Action (a):** The robot chooses an action (e.g., "Go Left").
4. **Next State (s'):** As a result of the action, the robot transitions to a new state (e.g., state 3).

This tuple $(s, a, R(s), s')$ —Current State, Action, Reward, Next State—is the fundamental building block of RL algorithms.

Example Trajectories

- **Option A (Go Left):**
 - Start at 4 ($R = 0$) $\rightarrow$ Go Left $\rightarrow$ At 3 ($R = 0$) $\rightarrow$ Go Left $\rightarrow$ At 2 ($R = 0$) $\rightarrow$ Go Left $\rightarrow$ At 1 ($R = 100$).
 - **Total Reward:** 100. (Plus it reached the best target).

- **Option B (Go Right):**

- Start at 4 ($R = 0$) $\rightarrow$ Go Right $\rightarrow$ At 5 ($R = 0$) $\rightarrow$ Go Right $\rightarrow$ At 6 ($R = 40$).
- **Total Reward:** 40. (Reached a target, but a less valuable one).

- **Option C (Wasted Motion):**

- Start at 4 $\rightarrow$ Go Right (to 5) $\rightarrow$ Go Left (back to 4) $\rightarrow$ Go Left (to 3) ... $\rightarrow$ At 1 ($R = 100$).
- **Result:** Still gets +100 reward eventually, but wasted time and fuel.

The goal of the algorithm is to figure out that **Option A** is the best strategy.

The Return in Reinforcement Learning 💰

In Reinforcement Learning, the goal isn't just to get a single reward, but to maximize the total sum of rewards over time. However, getting a reward *now* is usually better than getting the same reward *later*.

The Analogy: 5*Now* vs. 10 *Later*

- Imagine a \$5 bill is at your feet. You can pick it up instantly.
 - Imagine a \$10 bill is across town (30-minute walk).
 - Which do you choose?
 - The \$10 is larger, but requires effort and time.
 - If the effort/delay is too high, the \$5 bill becomes more attractive.
 - **The Concept:** Rewards obtained quickly are more valuable than rewards that take a long time to achieve.
-

Defining the Return

The **Return** is the sum of future rewards, but weighted by a **Discount Factor** (γ , **Gamma**).

- **Formula:**
If the sequence of rewards is $R_1, R_2, R_3, \dots$
$$\text{Return} = R_1 + \gamma R_2 + \gamma^2 R_3 + \gamma^3 R_4 + \dots$$
 - **The Discount Factor (γ):**
 - A number between 0 and 1 (usually close to 1, like 0.9 or 0.99).
 - It determines how "impatient" the algorithm is.
 - $\gamma^0 = 1$: The first reward gets 100% credit.
 - $\gamma^1 = 0.9$: The second reward gets 90% credit.
 - $\gamma^2 = 0.81$: The third reward gets 81% credit.
-

Example Calculation (Mars Rover)

Assume $\gamma = 0.5$ (very impatient).

Start at State 4.

Goal: Reach State 1 (Reward 100).

Path: $4 \rightarrow 3 \rightarrow 2 \rightarrow 1$.

- **Step 1 (State 4):** Reward = 0
- **Step 2 (State 3):** Reward = 0

- **Step 3 (State 2):** Reward = 0
- **Step 4 (State 1):** Reward = 100 (Terminal)

Calculate Return:

$$\text{Return} = 0 + (0.5 \times 0) + (0.5^2 \times 0) + (0.5^3 \times 100)$$

$$\text{Return} = 0 + 0 + 0 + (0.125 \times 100) = \mathbf{12.5}$$

- Even though the final reward is 100, because it is 3 steps away and $\gamma = 0.5$, its *present value* (Return) is only 12.5.
-

Comparing Policies

The return depends heavily on the **actions** the robot chooses.

- **Scenario A: Always Go Left** (Toward the 100 reward)
 - Start at State 4 $\rightarrow$ Return = **12.5**
 - Start at State 5 $\rightarrow$ Return = **6.25** (Farther away)
 - **Scenario B: Always Go Right** (Toward the 40 reward)
 - Start at State 4 $\rightarrow$ Return = **10**
 - Start at State 5 $\rightarrow$ Return = **20**
 - **Scenario C: Mixed Strategy**
 - If at State 4, go Left (Return 12.5 > 10).
 - If at State 5, go Right (Return 20 > 6.25).
 - **Insight:** The optimal strategy changes based on your current state to maximize the return.
-

Negative Rewards

What if rewards are negative (punishments)?

- The discount factor incentivizes the agent to **delay** the punishment as long as possible.
- *Analogy:* Paying a 10 fine today hurts more than paying a 10 fine 5 years from now (due to inflation/interest). The algorithm tries to push the negative event into the distant future.

The Policy (π)

In Reinforcement Learning, we need a formal way to define *how* an agent chooses an action based on its current situation. This function is called the **Policy**.

Definition

A Policy, denoted by the Greek letter **Pi** (π), is a function that maps a **State** (s) to an **Action** (a).

$$a = \pi(s)$$

- **Input:** The current State s .
- **Output:** The Action a that the agent should take.

The Goal of Reinforcement Learning

The ultimate goal of any RL algorithm is to find the **Optimal Policy** (often denoted as π^*).

- We are not just looking for *any* policy.
- We want to find the specific policy that, when followed, results in the **maximum possible Return** (sum of discounted rewards).

Example: Mars Rover Policy

Imagine a policy designed to grab the nearest reward:

Current State (s)	Policy Output $\pi(s)$	Meaning
State 2	Left	"If I am in State 2, go Left."
State 3	Left	"If I am in State 3, go Left."
State 4	Left	"If I am in State 4, go Left."
State 5	Right	"If I am in State 5, go Right."

In this example, the policy π is simply a lookup table or a set of rules telling the robot exactly what to do in every possible location.

Terminology Note

- In control theory or robotics, this might be called a "**Controller**."
- In Reinforcement Learning, the standard term is always "**Policy**."

Review: Key Concepts of Reinforcement Learning

We have established a standardized formalism to describe any Reinforcement Learning problem, regardless of whether it is a simple Mars Rover or a complex autonomous helicopter.

The Core Components

Concept	Symbol	Definition
State	s	The current situation or position of the agent (e.g., Rover is in box 4).
Action	a	What the agent decides to do (e.g., Move Left).
Reward	$R(s)$	The immediate feedback received for being in a state (e.g., +100 for finding science, 0 for empty rock).
Discount Factor	γ	A number between 0 and 1 (e.g., 0.99). It determines how much we value future rewards compared to immediate rewards.
Return		The total sum of discounted future rewards. The goal is to maximize this.
Policy	$\pi(s)$	The function (or "brain") that maps a state to an action. It tells the agent <i>what to do</i> in any given situation.

The Markov Decision Process (MDP)

This entire formalism—States, Actions, Rewards, and the transition logic—is collectively called a **Markov Decision Process (MDP)**.

The "Markov" Property

The term "Markov" implies a specific property about the system:

- **The future depends *only* on the current state.**
- It does **not** depend on the history of how you got there.
- *Example:* In chess, to decide the next best move, you only need to look at the current board. It does not matter *how* the pieces arrived at those positions; the history is irrelevant to the decision-making process.

The MDP Cycle

The MDP represents a continuous loop of interaction:

1. **Agent** observes the current state (s).
2. **Agent** uses its Policy (π) to choose an Action (a).
3. **Environment** reacts to the action.
4. **Environment** returns a new State (s') and a Reward (R) to the Agent.
5. Repeat.

Applications of the Formalism

The exact same mathematical framework applies to vastly different real-world problems.

Application	State (s)	Action (a)	Reward (R)
Mars Rover	Position (1-6)	Left, Right	+100 (Science), 0 (Nothing)
Helicopter	Position, Orientation, Speed, Rotation	Stick movements	+1 (Stable), -1000 (Crash)
Chess	Position of all pieces on the board	Legal moves	+1 (Win), -1 (Loss), 0 (Tie)

Note on Discount Factors:

- **Rover Example:** We used $\gamma = 0.5$ (very impatient) for illustration.
- **Real World:** We usually use $\gamma \approx 0.99$ or 0.999 (patient), as we care about the long-term outcome (winning the game or keeping the helicopter flying).